```
sample = "this is my code"
sample.chop  # => "this is my cod"
sample       # => "this is my code"

sample.chop! # => "this is my cod"
sample       # => "this is my cod"
```

Methods that can appear on the left side of an assignment (a feature we discussed back in the chapter on classes on page 38) end with an equal sign (=).

?, !, and = are the only weird characters allowed as method name suffixes. These characters are only allowed at the end of a method name.

In addition, there is a limited set of operators that you can override by defining them as methods. For example:

```
class Matrix
  attr_reader :x, :y

  def initialize(x, y)
    @x = x
    @y = y
  end

  def to_s = "(#{x}, #{y})"

  def +(other)
    Matrix.new(x + other.x, y + other.y)
  end
end

first = Matrix.new(1, 2)
second = Matrix.new(3, 4)
puts first + second
```

produces:

```
(4, 6)
```

Here we are defining the + operator to implement matrix addition and return a new matrix object. Even though the + is defined as a method, it is still written as a binary operator. The full list of operator names that you can define as methods is in Chapter 25, Language Reference: Objects and Classes, on page 465

Method Receiver

An instance method definition, like the ones we just saw, adds the method to the class it is defined within and makes the method available to instances of that class. (We'll talk in Chapter 25, Language Reference: Objects and Classes, on page 465 about what Ruby does for method definitions that are not inside a class.)

Ruby also allows you to define a method for one specific object, rather than the current class. The most common use of this feature is to assign methods to the class itself, rather than to instances of the class.

The syntax is to put the object name, followed by a dot, before the method name:

```
class Computer
  def self.function
```